

# CSCI 112

## Programming with C

### LECTURE: 14

J. L. Pach

# OUTLINE:

- What is a compound assignment?
- Arithmetic: `+=`, `-=`, `*=`, `/=`, `%=`
- Bitwise: `<<=`, `>>=`, `&=`, `^=`, `|=`
- Bit flags in practice
- Precedence & associativity
- Common mistakes
- Unions, Enums, Bit-fields

# THE BIG PICTURE

- A compound assignment combines an **operation** and an **assignment** in one operator.
- General form:

`x op= y` ≡ `x = x op (y)`

- `op` can be any of: `+ - * / % << >> & ^ |`
- The right-hand side is **always evaluated first, as a whole** (notice the parentheses!).
- The left operand is evaluated **only once**.

# ARITHMETIC

+=  -=  *=  /=  %=

# ARITHMETIC COMPOUND OPERATORS

Starting values for **each** row: `int x = 10, y = 3;`

| Operator | Code | Equivalent to | Result (`x`) |
|---|---|---|---|
| `+=` | `x += y;` | `x = x + y;` | `13` |
| `-=` | `x -= y;` | `x = x - y;` | `7` |
| `*=` | `x *= y;` | `x = x * y;` | `30` |
| `/=` | `x /= y;` | `x = x / y;` | `3` (integer division!) |
| `%=` | `x %= y;` | `x = x % y;` | `1` (remainder) |

# EXAMPLE - STEP BY STEP

```c
#include <stdio.h>

int main()
{
    int x = 10;

    x += 5;   printf("%d ", x);
    x -= 3;   printf("%d ", x);
    x *= 2;   printf("%d ", x);
    x /= 5;   printf("%d ", x);
    x %= 3;   printf("%d ", x);

    return 0;
}
```

Trace:

x = 10

x += 5 → 10 + 5 = 15

x -= 3 → 15 - 3 = 12

x *= 2 → 12 * 2 = 24

x /= 5 → 24 / 5 = 4 (not 4.8!)

x %= 3 → 4 % 3 = 1

Result:

15 12 24 4 1

# ⚠ THE RIGHT SIDE IS EVALUATED FIRST

```c
int x = 2, y = 3;

x *= y + 1;
printf("%d\n", x);
```

Result:
8

What the compiler does:

```
x *= y + 1;
x  = x * (y + 1);     ✅ 2 * 4 = 8

NOT:
x  = x * y + 1;       ❌ 2 * 3 + 1 = 7
```

- `x op= expression` → the **whole** expression on the right is computed first.
- Only then is the result combined with `x`.

# TYPE MATTERS (AGAIN!)

Just like `*`, `==` and `+`, the result of a compound assignment depends on the **type of the left operand**. The result is converted back to that type.

| Code | Computation | Stored in `x` |
|---|---|---|
| `int x = 7; x /= 2;` | `7 / 2` → int division | `3` |
| `int x = 7; x /= 2.0;` | `7 / 2.0 = 3.5` → converted to `int` | `3` |
| `int x = 7; x *= 1.5;` | `7 * 1.5 = 10.5` → converted to `int` | `10` |
| `double d = 7; d /= 2;` | `7.0 / 2 = 3.5` | `3.5` |
| `float f = 7; f %= 2;` | `%` needs integers | ❌ compile error |

# BITWISE

<<=  >>=  &=  ^=  |=

# BITWISE COMPOUND OPERATORS

Starting values for **each** row: `unsigned int x = 12, y = 10;`

`x = 0000 1100` `y = 0000 1010`

| Operator | Code | Equivalent to | Binary result | Result (`x`) |
|---|---|---|---|---|
| <<= | x <<= 2; | x = x << 2; | 0011 0000 | 48 |
| >>= | x >>= 2; | x = x >> 2; | 0000 0011 | 3 |
| &= | x &= y; | x = x & y; | 0000 1000 | 8 |
| ^= | x ^= y; | x = x ^ y; | 0000 0110 | 6 |
| \|= | x \|= y; | x = x \| y; | 0000 1110 | 14 |

# EXAMPLE - STEP BY STEP

```c
#include <stdio.h>

int main()
{
    unsigned int x = 5;

    x <<= 1;  printf("%u ", x);
    x <<= 2;  printf("%u ", x);
    x >>= 3;  printf("%u ", x);
    x |= 8;   printf("%u ", x);
    x &= 12;  printf("%u ", x);
    x ^= 15;  printf("%u ", x);

    return 0;
}
```

Trace:

x = 0101 (5)

x <<= 1 → 1010 (10)

x <<= 2 → 101000 (40)

x >>= 3 → 0101 (5)

x |= 8 → 1101 (13)

x &= 12 → 1100 (12)

x ^= 15 → 0011 (3)

Result:

10 40 5 13 12 3

# SHIFTS AS FAST MULTIPLICATION / DIVISION

```c
unsigned int x = 3;

x <<= 1;   // x = 3 * 2  = 6
x <<= 3;   // x = 6 * 8  = 48
x >>= 2;   // x = 48 / 4 = 12
```

```
x <<= n   ≡   x *= 2^n
x >>= n   ≡   x /= 2^n
```

- This works reliably for **unsigned** (or non-negative) values.
- ⚠️ Shifting by `n >=` the number of bits in the type (e.g. `x <<= 32` for a 32-bit `int`) is **undefined behavior**.
- ⚠️ Left-shifting a **negative** signed value is **undefined behavior**.

# BIT FLAGS

## in practice

# THE FOUR FLAG OPERATIONS

Each bit in a variable can represent an independent **on/off** option (a *flag*).

```c
#define READ   (1 << 0)   // 0000 0001
#define WRITE  (1 << 1)   // 0000 0010
#define EXEC   (1 << 2)   // 0000 0100
```



| Goal | Code | Why it works |
|---|---|---|
| **Set** a flag | `flags \|= WRITE;` | `x \| 1 = 1`, `x \| 0 = x` |
| **Clear** a flag | `flags &= ~WRITE;` | `~WRITE = 1111 1101`, so only that bit becomes `0` |
| **Toggle** a flag | `flags ^= WRITE;` | `x ^ 1 = !x`, `x ^ 0 = x` |
| **Check** a flag | `if (flags & WRITE)` | result is non-zero only if the bit is set |

# EXAMPLE - PERMISSIONS

```c
#include <stdio.h>

#define READ   (1 << 0)
#define WRITE  (1 << 1)
#define EXEC   (1 << 2)

int main()
{
    unsigned char flags = 0;

    flags |= READ;     // set
    flags |= WRITE;    // set
    flags &= ~READ;    // clear
    flags ^= EXEC;     // toggle

    if (flags & WRITE)
        printf("can write\n");
    if (!(flags & READ))
        printf("cannot read\n");

    printf("flags = %d\n", flags);
    return 0;
}
```

Trace:

flags = 0000 0000 (0)

|= READ → 0000 0001 (1)

|= WRITE → 0000 0011 (3)

&= ~READ → 0000 0010 (2)

^= EXEC → 0000 0110 (6)

Result:

can write

cannot read

flags = 6

# PRECEDENCE

## & associativity

# WHERE DO COMPOUND OPERATORS FIT?

| Priority | Operators | Associativity |
|---|---|---|
| highest | `()` `[]` `->` `.` `x++` `x--` | left → right |
| | `++x` `--x` `!` `~` `*ptr` `&x` `(type)` `sizeof` | right → left |
| | `*` `/` `%` then `+` `-` then `<<` `>>` | left → right |
| | `<` `<=` `>` `>=` then `==` `!=` | left → right |
| | `&` then `^` then `\|` then `&&` then `\|\|` | left → right |
| | `?:` | right → left |
| **very low** | `=` `+=` `-=` `*=` `/=` `%=` `<<=` `>>=` `&=` `^=` `\|=` | **right → left** |
| lowest | `,` | left → right |

- Almost everything on the right side is evaluated **before** the assignment happens.

# RIGHT-TO-LEFT ASSOCIATIVITY

```c
int a = 1, b = 2, c = 3;

a += b += c;

printf("%d %d %d\n", a, b, c);
```

Result:

```
6 5 3
```

Grouped right to left:

```
a += (b += c);
```

1) b += c → b = 2 + 3 = 5
2) a += 5 → a = 1 + 5 = 6

- Like `=`, a compound assignment is an **expression**: its value is the new value of the left operand.
- It is legal, but **chaining makes code hard to read** → write one assignment per line.

# COMMON

## mistakes

# COMMON MISTAKES

| ❌ Wrong | What happens | ✅ Correct |
|---|---|---|
| `x =+ 5;` | parsed as `x = +5;` → `x` becomes `5` | `x += 5;` |
| `x + = 5;` | space inside the operator → compile error | `x += 5;` |
| `int sum; sum += x;` | `sum` is uninitialized → garbage result | `int sum = 0; sum += x;` |
| `int x = 7; x /= 2;` (expecting `3.5`) | integer division → `3` | `double x = 7; x /= 2;` |
| `x *= y + 1;` (expecting `x*y + 1`) | right side evaluated first → `x * (y + 1)` | `x = x * y + 1;` |
| `float f; f %= 2;` | `%` works only on integers → compile error | `f = fmodf(f, 2);` (`<math.h>`) |
| `flags & ~WRITE;` | result computed and thrown away, `flags` unchanged | `flags &= ~WRITE;` |
| `flags &= WRITE;` (to clear) | clears **all other** bits, keeps only `WRITE` | `flags &= ~WRITE;` |
| `i += i++;` | `i` modified twice in one expression → **undefined behavior** | `i++; i += i;` (pick one meaning) |
| `x <<= 32;` (32-bit `int`) | shift ≥ width of type → **undefined behavior** | shift by `0 … 31` only |

# ⚠ =+ VS +=

```c
#include <stdio.h>

int main()
{
    int a = 10, b = 10;

    a += 5;
    b =+ 5;

    printf("a = %d\n", a);
    printf("b = %d\n", b);

    return 0;
}
```

How the compiler reads it:

a += 5; → a = a + 5; ✅

b =+ 5; → b = (+5); ❌
(unary plus!)

Result:

```
a = 15
b = 5
```

# ⚠ FORGETTING TO INITIALIZE

```c
int arr[] = {1, 2, 3, 4};
int sum;            // ❌ garbage

for (int i = 0; i < 4; i++)
{
    sum += arr[i];
}
```



```c
int arr[] = {1, 2, 3, 4};
int sum = 0;        // ✅

for (int i = 0; i < 4; i++)
{
    sum += arr[i];
}
```



- `x op= y` **reads** `x` before writing it. The variable must already hold a meaningful value.
- Accumulators: start with `0` for `+=` and `1` for `*=`.
- Compile with `-Wall` and the compiler will often warn you: *"'sum' is used uninitialized"*.

# SUMMARY

| Rule | Example |
| --- | --- |
| `x op= y` ≡ `x = x op (y)` | `x *= y + 1;` → `x = x * (y + 1);` |
| Arithmetic: `+=` `-=` `*=` `/=` `%=` | `sum += arr[i];` |
| Bitwise: `<<=` `>>=` `&=` `^=` `\|=` | `x <<= 1;` → `x *= 2` |
| The result is converted to the **type of the left operand** | `int x = 7; x *= 1.5;` → `10` |
| `%=` works only on integers | `float f; f %= 2;` ❌ |
| Set / clear / toggle / check a flag | `\|= F` `&= ~F` `^= F` `& F` |
| Very low precedence, **right → left** | `a += b += c;` → `a += (b += c);` |
| The operator symbol comes **before** `=` | `+=` ✅ `=+` ❌ |
| Initialize before using `op=` | `int sum = 0;` |
| Never modify a variable twice in one expression | `i += i++;` ❌ UB |

# UNIONS

# An example - unions

```c
struct Example1
{
    char c;
    int i;
    short s;
};
```



```c
union Example2
{
    short s;
    char c;
    int i;
};
```



```c
printf( "Size of a struct Example is = %d\n", sizeof(struct Example1) );
// Result: Size of a struct Example is = 12
printf( "Size of a struct Example is = %d\n", sizeof(union Example2) ); // Result: Size of a struct Example is = 4
struct Example1 example1;
printf( "Struct Example1:\n" ); // Result: Struct Example1:
printf( "Address of variable c = %d\n", &example1.c ); // Result: Address of variable c = 6487828
printf( "Address of variable i = %d\n", &example1.i ); // Result: Address of variable i = 6487832
printf( "Address of variable s = %d\n", &example1.s ); // Result: Address of variable s = 6487836
union Example2 example2;
printf( "Union Example2:\n" );  // Result: Union Example2:
printf( "Address of variable s = %d\n", &example2.s ); // Result: Address of variable s = 6487824
printf( "Address of variable c = %d\n", &example2.c ); // Result: Address of variable c = 6487824
printf( "Address of variable i = %d\n", &example2.i ); // Result: Address of variable i = 6487824
```



# UNIONS

- Unions in C are special data types that allow different data types to be stored in the same memory location.

Syntax:

```
unions symbolic_name1
{
        <statement1>
}<symbolic_name2, ...>;
```



- Everything that is in angle brackets `<>` is optional.

# An example - unions

```c
#include <stdio.h>
union Example
{
    short s;
    char c;
    int i;
};
```



Unions in C are a mechanism that allows different data types to be stored in the same memory location, which can be useful in specific situations but requires caution due to potential pitfalls associated with their use.

```c
union Example example;
printf( "Union Example:\n" );// Result: Union Example:
example.s = 5;
printf( "The value of [example.s] = %d\n", example.s );// Result: The value of [example.s] = 5
example.c = 'A';
printf( "The value of [example.c] = %d\n", example.c );// Result: The value of [example.c] = 65
example.i = 63123;
printf( "The value of [example.i] = %d\n", example.i );// Result: The value of [example.i] = 63123
/*but...*/
printf( "The value of [example.s] = %d\n", example.s );// Result: The value of [example.s] = -2413
```



# A SUMMARY ON UNIONS:

- Unions in C are special data types that allow different data types to be stored in the same memory location. Unlike structures, where each member has its own dedicated space, all members of a union share the same location. This means that at any given time, only one member of the union can hold a defined value, and changing the value of one member automatically overwrites the values of the others.
- This makes unions ideal for situations where we need to store different types of data in the same place, but only one of these types is active at any moment. Unions have various applications, from representing variable data types to creating more memory-efficient data structures. However, it's important to note that improper use of unions can lead to programming errors, such as accessing uninitialized data or violating memory alignment rules.

# ENUMS

# ENUMS

- An enum (enumeration) is a user-defined data type in C that consists of a set of named integer constants. These constants are often used to represent a fixed set of values.
- Enum syntax is the same as struct.

Syntax:

```
enums symbolic_name1
{
        <statement1>
}<symbolic_name2, ...>;
```



- Everything that is in angle brackets <> is optional.

# 1 example

```c
#include <stdio.h>
enum color
{
    red,
    green,
    blue,
};
```



These constants are often used to represent a fixed set of values, such as days of the week, colors, or error codes. Enums provide a way to make code more readable and maintainable by using meaningful names instead of raw integer values.

```c
printf("The size of [enum color] is %d\n", sizeof(enum color));   /* default as sizeof(int) */
enum color my_color = red;           // Result: The size of [enum color] is 4
printf("The value of my_color is %d\n", my_color); // Result: The value of my_color is 0
int x = green;
printf("The value of x is %d\n", x); // Result: The value of x is 1
my_color = 99;  /* be careful */
printf("The value of my_color is %d\n", my_color); // Result: The value of my_color is 99
if(my_color == red || my_color == green || my_color == blue )
  printf("Proper value\n"); else printf("Improper value\n"); // Result: Improper value
```



```c
#include <stdio.h>
enum color
{
    red = 51,
    green,
    blue = 91,
    orange,
};
```

When using elements from a defined enum, we don't need to use the enum's name itself. They are treated by the compiler as integer constants, which means they can be used to define array sizes just like the #define preprocessor directive.

```c
int array[red];
printf("The value of red is %d\n", red); // Result: The value of red is 51
printf("The value of green is %d\n", green); // Result: The value of green is 52
printf("The value of blue is %d\n", blue); // Result: The value of blue is 91
printf("The value of blue is %d\n", orange); // Result: The value of orange is 92
```

If we don't specify a value for an enumeration field, the default value is 0. The subsequent field will have a value one greater than the previous one. However, if we define a different value for a field, that field will have an individually defined value and the next field will be one more than the last one.

# ENUMS

- An enum (enumeration) is a user-defined data type in C that consists of a set of named integer constants. These constants are often used to represent a fixed set of values.
- Enum syntax is the same as struct.

Syntax:

```
enums symbolic_name1
{
        <statement1>
}<symbolic_name2, ...>;
```



- Everything that is in angle brackets <> is optional.

```c
#include <stdio.h>
enum color
{
    red = 51,
    green,
    blue = 91,
    orange,
}my_global_color, * my_pointer_color;
```



```c
enum color my_global = orange;
enum color my_local_color = red;
printf("The value of my_local_color is %d\n", my_local_color);
// Result: The value of my_local_color is 51
my_pointer_color = &my_local_color;
*my_pointer_color = blue;
```



```c
printf("The value of my_local_color is %d\n", my_local_color);
// Result: The value of my_local_color is 91
switch (my_global)
{
  case red:
    printf("red\n");
    break;
  case green:
    printf("green\n");
    break;
  case blue:
    printf("blue\n");
    break;
  case orange:
    printf("orange\n"); // Result: orange
    break;
  default:
    printf("none\n");
    break;
}
```



Given that enums can also create global variables within their definition (including pointers), it's important to keep this in mind. Enums are often used with switch statements, allowing us to convert numerical values into strings.

# BIT

## fields

# Example

```c
#include <stdio.h>
struct Data
{
   unsigned char even_number : 1;
   /* instead of using the char data type,
      if you use the int data type, it will occupy 4 bytes */
   unsigned char greater_than_10 : 1;
   unsigned char is_the_power_of_two : 1;
};
```



> A bit-field structure occupies as much space as the largest defined field type, instead of assigning a value `=` to fields, the number of BITS for this flag `:` is specified.

```c
   printf("The size of Data is %d\n", sizeof(struct Data)); // The size of Data is 1
   struct Data my_data;
   int value;
   printf("Give me a number\n"); // Give me a number
   _ = scanf("%d", &value);     // Case 1: 8  Case 2: 12 Case 3: 11
   my_data.even_number = !(value % 2);
   my_data.greater_than_10 = ( value > 10 );
   my_data.is_the_power_of_two = value && !(value & (value - 1));
   printf("%d %d %d \n", my_data.even_number, my_data.greater_than_10, my_data.is_the_power_of_two );
   // Result1: 1 0 1 or Result2: 1 1 0 Result3: 0 1 0
```



# THANK

**You!**